

Lesson 3 Structured Note: Array Stack Operations

0. Current Position in the Course

You are now in **Lesson 3** of the stack curriculum.

This lesson belongs to:

Phase 1: Foundations and Array Stack

In the previous lessons, you learned:

- A stack follows **LIFO**: Last In, First Out.
- Insertion and deletion happen only at the **top**.
- An array stack stores stack elements inside an array.
- An array stack uses three important fields:

```
int size;  
int top;  
int *S;
```

You also learned these two important conditions:

```
Empty condition: top == -1  
Full condition:  top == size - 1
```

Now Lesson 3 teaches how to perform the main operations on an array stack.

1. Lesson 3 Goal

By the end of this lesson, you should understand how to implement and dry-run these array stack operations:

```
push()  
pop()  
peek()  
stackTop()  
isEmpty()  
isFull()
```

You should also understand:

- Which operations change the stack.
- Which operations only read the stack.
- Why some functions receive the stack by pointer.
- Why some functions can receive the stack by value.
- How to detect overflow.
- How to detect underflow.
- How to calculate the correct array index for `peek(pos)`.
- Why all these array stack operations are $O(1)$.

2. Stack Structure Used in Lesson 3

The array stack uses this structure:

```
struct Stack {  
    int size;  
    int top;  
    int *S;  
};
```

Meaning of Each Field

Field	Simple Meaning
<code>size</code>	Maximum number of elements the stack can store
<code>top</code>	Index of the current top element
<code>S</code>	Pointer to the array where stack values are stored

Example:

```
Index: 0    1    2  
Value: 10   20   30  
  
top = 2
```

Here:

```
S[top] = S[2] = 30
```

So the top element is `30`.

Important point:

```
top stores the index of the top element.  
It does not store the actual value.
```

Wrong idea:

```
top = 30
```

Correct idea:

```
top = 2  
S[top] = 30
```

3. Review: Empty and Full Conditions

Before learning the operations, remember these two formulas.

Empty Condition

An array stack is empty when:

```
top == -1
```

This means no valid array index is active.

Example:

```
Index: 0    1    2    3    4  
Value: [ ] [ ] [ ] [ ] [ ]  
  
top = -1
```

`top = -1` does not mean `S[-1]` exists.

It is only a marker meaning:

The stack is empty.

Full Condition

An array stack is full when:

$\text{top} == \text{size} - 1$

Example:

$\text{size} = 5$

Valid indices are:

0, 1, 2, 3, 4

The last valid index is:

$\text{size} - 1 = 5 - 1 = 4$

So the stack is full when:

$\text{top} == 4$

4. Operation 1: `isEmpty()`

Meaning

`isEmpty()` checks whether the stack has no active elements.

It answers this question:

Is the stack empty?

For an array stack, the stack is empty when:

```
top == -1
```

Code Version 1

```
int isEmpty(struct Stack st) {  
    if (st.top == -1)  
        return 1;  
    else  
        return 0;  
}
```

Code Version 2: Shorter Version

```
int isEmpty(struct Stack st) {  
    return st.top == -1;  
}
```

Both versions mean the same thing.

Return Values

Return Value	Meaning
1	Stack is empty
0	Stack is not empty

Example 1

Given:

```
top = -1
```

Then:

```
isEmpty(st)
```

returns:

```
1
```

Because the stack is empty.

Example 2

Given:

```
top = 2
```

Then:

```
isEmpty(st)
```

returns:

```
0
```

Because the stack has active elements.

Does `isEmpty()` Change the Stack?

No.

It only reads `top`.

So it can receive the stack by value:

```
struct Stack st
```

5. Operation 2: `isFull()`

Meaning

`isFull()` checks whether the stack has reached its maximum capacity.

It answers this question:

Is the stack full?

For an array stack, the stack is full when:

`top == size - 1`

Code Version 1

```
int isFull(struct Stack st) {  
    if (st.top == st.size - 1)  
        return 1;  
    else  
        return 0;  
}
```

Code Version 2: Shorter Version

```
int isFull(struct Stack st) {  
    return st.top == st.size - 1;  
}
```

Return Values

Return Value	Meaning
1	Stack is full
0	Stack is not full

Example 1

Given:

```
size = 5  
top = 4
```

Check:

```
top == size - 1  
4 == 5 - 1  
4 == 4
```

This is true.

So:

```
isFull(st)
```

returns:

```
1
```

Example 2

Given:

```
size = 5  
top = 2
```

Check:

```
top == size - 1  
2 == 5 - 1  
2 == 4
```

This is false.

So:

```
isFull(st)
```

returns:

```
0
```

Does `isFull()` Change the Stack?

No.

It only reads `top` and `size`.

So it can receive the stack by value:

```
struct Stack st
```

6. Operation 3: `push(x)`

Meaning

`push(x)` inserts a new value at the top of the stack.

Example:

Before `push(40)`:

Index:	0	1	2
Value:	10	20	30

`top = 2`

After:

```
push(40)
```

The stack becomes:

Index:	0	1	2	3
Value:	10	20	30	40

`top = 3`

The new top element is `40`.

Main Idea

A successful `push()` does two things:

1. Increase `top` by 1.
2. Store the new value at `S[top]`.

In code:

```
top++;  
S[top] = x;
```

Important Safety Check

Before pushing, we must check whether the stack is full.

If the stack is already full, we cannot insert another value.

That error is called:

Stack Overflow

Overflow Condition

```
st->top == st->size - 1
```

If this condition is true, the stack is full.

Code

```
void push(struct Stack *st, int x) {  
    if (st->top == st->size - 1) {  
        printf("Stack Overflow\n");  
    } else {  
        st->top++;  
        st->S[st->top] = x;  
    }  
}
```

Why Use `struct Stack *st`?

`push()` changes the stack.

It changes:

```
st->top  
st->S[st->top]
```

Because the function must change the original stack, we pass the stack by address:

```
struct Stack *st
```

That is why we use the arrow operator:

```
st->top  
st->size  
st->S
```

`.` **vs** `->`

When you have a normal structure variable, use dot:

```
st.top
```

When you have a pointer to a structure, use arrow:

```
st->top
```

In `push()`, `st` is a pointer, so we use:

```
st->top
```

7. `push()` Dry Run

Given:

```
size = 5  
top = 3
```

Current stack:

```
Index: 0    1    2    3    4  
Value: 10   16   20   25   empty  
  
top = 3
```

Now call:

```
push(&st, 30);
```

Step 1: Check Whether the Stack Is Full

Full condition:

```
top == size - 1
```

Substitute the values:

```
3 == 5 - 1  
3 == 4
```

This is false.

So the stack is not full.

Step 2: Increase `top`

```
st->top++;
```

So:

```
top: 3 -> 4
```

Step 3: Store the New Value

```
st->S[st->top] = 30;
```

Since `top = 4`:

```
S[4] = 30
```

Final stack:

```
Index: 0    1    2    3    4
Value: 10   16   20   25   30

top = 4
```

Now the stack is full.

Try One More Push

Now call:

```
push(&st, 40);
```

Check full condition:

```
top == size - 1
4 == 5 - 1
4 == 4
```

This is true.

So the function prints:

```
Stack Overflow
```

No value is inserted.

8. Operation 4: `pop()`

Meaning

`pop()` removes and returns the top element of the stack.

Example:

Before `pop()`:

Index:	0	1	2	3
Value:	10	20	30	40

`top = 3`

The top element is:

`S[top] = S[3] = 40`

After `pop()`:

Returned value = 40
`top = 2`

Logical stack after pop:

Top -> 30
 20
 10

Main Idea

A successful `pop()` does three things:

1. Save the value at `S[top]`.
2. Decrease `top` by 1.
3. Return the saved value.

In code:

```
x = S[top];  
top--;  
return x;
```

Important Safety Check

Before popping, we must check whether the stack is empty.

If the stack is empty, there is no value to remove.

That error is called:

Stack Underflow

Underflow Condition

```
st->top == -1
```

If this condition is true, the stack is empty.

Code

```
int pop(struct Stack *st) {  
    int x = -1;  
  
    if (st->top == -1) {  
        printf("Stack Underflow\n");  
    } else {  
        x = st->S[st->top];  
        st->top--;  
    }  
  
    return x;  
}
```

Why Is `x` Initialized to `-1`?

The variable `x` stores the value that will be returned.

```
int x = -1;
```

If the stack is empty, there is no real stack value to return.

So `-1` is used as an error value.

If the pop is successful, `x` is replaced with the actual top value.

Example:

```
x = st->S[st->top];
```

Why Use `struct Stack *st`?

`pop()` changes the stack because it decreases `top`.

So it must receive the real stack address:

```
struct Stack *st
```

9. `pop()` Dry Run

Start:

```
Index: 0    1    2    3    4
Value: 10   16   20   25   30

top = 4
```

Call:

```
pop(&st);
```

Step 1: Check Whether the Stack Is Empty

Empty condition:

```
top == -1
```

Substitute the value:

```
4 == -1
```

This is false.

So the stack is not empty.

Step 2: Save the Top Value

```
x = st->S[st->top];
```

Since `top = 4`:

```
x = S[4]  
x = 30
```

Step 3: Decrease `top`

```
st->top--;
```

So:

```
top: 4 -> 3
```

Step 4: Return the Saved Value

```
pop() returns 30
```

Final logical stack:

```
Top -> 25  
      20
```

```
16
10
```

Important Memory Note

The value `30` may still physically remain in the array at index `4`.

But it is no longer part of the logical stack because:

```
top = 3
```

The active stack is only:

```
10, 16, 20, 25
```

Index `4` is ignored.

10. Operation 5: `peek(pos)`

Meaning

`peek(pos)` reads a value at a given position from the top without deleting it.

It answers this question:

```
What value is at position pos from the top?
```

The position is counted from the top.

Example:

```
Top -> 40   position 1
        30   position 2
        20   position 3
        10   position 4
```

So:

```
peek(1) returns 40  
peek(2) returns 30  
peek(3) returns 20  
peek(4) returns 10
```

Important Point

`peek()` only reads.

It does not remove anything.

So after calling `peek()`, the stack remains unchanged.

Why Do We Need a Formula?

The user gives a position from the top.

But the array uses indices from the left side.

So we need to convert:

Position from top -> Array index

Peek Formula

$$\text{index} = \text{top} - \text{pos} + 1$$

This formula is very important.

Why the Formula Works

Suppose:

`top = 3`

That means the top element is at index `3`.

For `peek(1)`:

```
index = top - pos + 1
index = 3 - 1 + 1
index = 3
```

So `peek(1)` returns `S[3]`, which is the top element.

For `peek(2)`:

```
index = 3 - 2 + 1
index = 2
```

So `peek(2)` returns `S[2]`.

For `peek(3)`:

```
index = 3 - 3 + 1
index = 1
```

So `peek(3)` returns `S[1]`.

Code

```
int peek(struct Stack st, int pos) {
    int index = st.top - pos + 1;

    if (index < 0) {
        printf("Invalid Position\n");
        return -1;
    } else {
        return st.S[index];
    }
}
```

Stronger Code Version

A stronger version also checks whether `pos <= 0`.

This matters because positions should start from `1`, not `0` or negative numbers.

```
int peek(struct Stack st, int pos) {
    int index = st.top - pos + 1;
```

```

    if (pos <= 0 || index < 0) {
        printf("Invalid Position\n");
        return -1;
    }

    return st.S[index];
}

```

Does peek() Change the Stack?

No.

It only reads a value.

So it can receive the stack by value:

```
struct Stack st
```

11. peek() Dry Run

Given:

```

Index: 0    1    2    3
Value: 10   16   20   25

top = 3

```

Logical stack view:

```

Top -> 25   position 1
       20   position 2
       16   position 3
       10   position 4

```

Call:

```
peek(st, 3);
```

Use the formula:

```
index = top - pos + 1
```

Substitute:

```
index = 3 - 3 + 1  
index = 1
```

So:

```
peek(3) returns S[1]
```

Since:

```
S[1] = 16
```

Final answer:

```
peek(3) returns 16
```

The stack does not change.

12. More `peek()` Examples

Given:

```
Index: 0    1    2    3  
Value: 10   16   20   25  
  
top = 3
```

Logical view:

```
Top -> 25   position 1  
      20   position 2
```

```
16 position 3
10 position 4
```

peek(1)

```
index = top - pos + 1
index = 3 - 1 + 1
index = 3
```

So:

peek(1) returns $S[3] = 25$

peek(2)

```
index = 3 - 2 + 1
index = 2
```

So:

peek(2) returns $S[2] = 20$

peek(4)

```
index = 3 - 4 + 1
index = 0
```

So:

peek(4) returns $S[0] = 10$

peek(5)

```
index = 3 - 5 + 1
index = -1
```

This is invalid because array index **-1** does not exist.

So the function returns:

-1

and may print:

Invalid Position

13. Operation 6: `stackTop()`

Meaning

`stackTop()` returns the current top element without deleting it.

It is like:

`peek(1)`

But it is simpler because it only checks the top element.

Code

```
int stackTop(struct Stack st) {  
    if (st.top == -1)  
        return -1;  
    else  
        return st.S[st.top];  
}
```

Shorter Version

```
int stackTop(struct Stack st) {  
    if (st.top == -1)  
        return -1;  
  
    return st.S[st.top];  
}
```

Example

Given:

```
Index: 0    1    2
Value: 10   20   30

top = 2
```

Call:

```
stackTop(st)
```

Since:

```
S[top] = S[2] = 30
```

The function returns:

```
30
```

The stack does not change.

Difference Between `pop()` and `stackTop()`

Operation	Returns Top Value?	Removes Top Value?	Changes <code>top</code> ?
<code>pop()</code>	Yes	Yes	Yes
<code>stackTop()</code>	Yes	No	No

14. Full Lesson 3 C Program

This program includes the main Lesson 3 operations.

```
#include <stdio.h>
#include <stdlib.h>

struct Stack {
```

```

    int size;
    int top;
    int *S;
};

void push(struct Stack *st, int x) {
    if (st->top == st->size - 1) {
        printf("Stack Overflow\n");
    } else {
        st->top++;
        st->S[st->top] = x;
    }
}

int pop(struct Stack *st) {
    int x = -1;

    if (st->top == -1) {
        printf("Stack Underflow\n");
    } else {
        x = st->S[st->top];
        st->top--;
    }

    return x;
}

int peek(struct Stack st, int pos) {
    int index = st.top - pos + 1;

    if (pos <= 0 || index < 0) {
        printf("Invalid Position\n");
        return -1;
    }

    return st.S[index];
}

int stackTop(struct Stack st) {
    if (st.top == -1)
        return -1;

    return st.S[st.top];
}

int isEmpty(struct Stack st) {
    return st.top == -1;
}

```

```

int isFull(struct Stack st) {
    return st.top == st.size - 1;
}

int main() {
    struct Stack st;

    printf("Enter stack size: ");
    scanf("%d", &st.size);

    st.S = (int *)malloc(st.size * sizeof(int));
    st.top = -1;

    push(&st, 10);
    push(&st, 20);
    push(&st, 30);

    printf("Top element: %d\n", stackTop(st));
    printf("Peek position 2: %d\n", peek(st, 2));
    printf("Popped: %d\n", pop(&st));
    printf("Top element after pop: %d\n", stackTop(st));

    if (isEmpty(st))
        printf("Stack is empty\n");
    else
        printf("Stack is not empty\n");

    if (isFull(st))
        printf("Stack is full\n");
    else
        printf("Stack is not full\n");

    free(st.S);

    return 0;
}

```

15. Explanation of the Full Program

Header Files

```
#include <stdio.h>
#include <stdlib.h>
```

`stdio.h` is used for:

```
printf()
scanf()
```

`stdlib.h` is used for:

```
malloc()
free()
```

Stack Declaration

```
struct Stack st;
```

This creates a stack variable named `st`.

Reading Stack Size

```
printf("Enter stack size: ");
scanf("%d", &st.size);
```

This asks the user to enter the maximum stack capacity.

Allocating Memory

```
st.S = (int *)malloc(st.size * sizeof(int));
```

This creates an integer array on the heap.

If `st.size = 5`, it creates space for 5 integers.

Initializing `top`

```
st.top = -1;
```

This marks the stack as empty.

Pushing Values

```
push(&st, 10);  
push(&st, 20);  
push(&st, 30);
```

These calls insert values into the stack.

After these calls:

```
Index: 0    1    2  
Value: 10   20   30
```

```
top = 2
```

Logical stack:

```
Top -> 30  
      20  
      10
```

Reading the Top Element

```
printf("Top element: %d\n", stackTop(st));
```

This prints:

```
30
```

Peeking Position 2

```
printf("Peek position 2: %d\n", peek(st, 2));
```

Logical stack:

```
Top -> 30   position 1
        20   position 2
        10   position 3
```

So `peek(2)` returns:

```
20
```

Popping

```
printf("Popped: %d\n", pop(&st));
```

This removes and returns `30`.

After popping:

```
Index: 0    1
Value: 10   20

top = 1
```

Logical stack:

```
Top -> 20
        10
```

Freeing Memory

```
free(st.S);
```

This releases the heap memory used by the stack array.

16. Parameter Passing in Lesson 3

Some functions receive the stack by pointer.

Some functions receive the stack by value.

This depends on whether the function changes the stack.

Summary Table

Operation	Changes the Stack?	Passing Style	Reason
<code>push()</code>	Yes	Pass by pointer	It changes <code>top</code> and writes into <code>S[top]</code>
<code>pop()</code>	Yes	Pass by pointer	It changes <code>top</code>
<code>peek()</code>	No	Pass by value	It only reads from the stack
<code>stackTop()</code>	No	Pass by value	It only reads the top value
<code>isEmpty()</code>	No	Pass by value	It only checks <code>top</code>
<code>isFull()</code>	No	Pass by value	It only checks <code>top</code> and <code>size</code>

Simple Rule

If the function changes the stack, pass by pointer.
If the function only reads the stack, pass by value.

Functions That Modify the Stack

```
push(struct Stack *st, int x)
pop(struct Stack *st)
```

These need the original stack address.

Functions That Only Read the Stack

```
peek(struct Stack st, int pos)
stackTop(struct Stack st)
isEmpty(struct Stack st)
isFull(struct Stack st)
```

These do not need to modify the original stack.

17. Operation Effects Summary

Operation	What It Does	Effect on <code>top</code>
<code>push(x)</code>	Inserts <code>x</code> at the top	Increases <code>top</code> by 1
<code>pop()</code>	Removes and returns top value	Decreases <code>top</code> by 1
<code>peek(pos)</code>	Reads position from top	No change
<code>stackTop()</code>	Reads current top	No change
<code>isEmpty()</code>	Checks if stack is empty	No change
<code>isFull()</code>	Checks if stack is full	No change

18. Dry Run: Full Example

Suppose:

```
size = 5
top = -1
```

Perform:

```
push(&st, 10);
push(&st, 20);
push(&st, 30);
peek(st, 2);
pop(&st);
stackTop(st);
```

Step-by-Step Trace

Step	Operation	top Before	top After	Return Value	Active Stack
0	Start	-1	-1	-	Empty
1	<code>push(10)</code>	-1	0	-	10

Step	Operation	top Before	top After	Return Value	Active Stack
2	push(20)	0	1	-	10, 20
3	push(30)	1	2	-	10, 20, 30
4	peek(2)	2	2	20	10, 20, 30
5	pop()	2	1	30	10, 20
6	stackTop()	1	1	20	10, 20

Final logical stack:

```
Top -> 20
      10
```

19. Required Dry Run 1: Push and Overflow

Given:

```
size = 5
top = 3
```

Current stack:

```
Index: 0    1    2    3    4
Value: 10   16   20   25   empty
```

Now call:

```
push(&st, 30);
```

Since:

```
3 != 5 - 1
3 != 4
```

The stack is not full.

After push:

```
Index: 0    1    2    3    4
Value: 10   16   20   25   30

top = 4
```

Now call:

```
push(&st, 40);
```

Since:

```
4 == 5 - 1
4 == 4
```

The stack is full.

Output:

```
Stack Overflow
```

20. Required Dry Run 2: Pop

Given:

```
Index: 0    1    2    3    4
Value: 10   16   20   25   30

top = 4
```

Call:

```
pop(&st);
```

Steps:

1. Check if `top == -1`.
2. It is false because `top = 4`.
3. Save `S[4]`, which is 30.
4. Decrease `top` from 4 to 3.
5. Return 30.

Result:

```
pop() returns 30
top = 3
```

Logical stack:

```
Top -> 25
      20
      16
      10
```

21. Required Dry Run 3: Peek

Given:

```
top = 3
```

Array:

Index:	0	1	2	3
Value:	10	16	20	25

Logical stack:

```
Top -> 25   position 1
      20   position 2
      16   position 3
      10   position 4
```

Call:

```
peek(st, 4);
```

Formula:

```
index = top - pos + 1
```

Substitute:

```
index = 3 - 4 + 1  
index = 0
```

So:

```
peek(4) returns S[0]
```

Since:

```
S[0] = 10
```

Final answer:

```
peek(4) returns 10
```

22. Complexity of Lesson 3 Operations

All individual array stack operations in Lesson 3 are $O(1)$ time.

Complexity Table

Operation	Time Complexity	Auxiliary Space	Reason
<code>push()</code>	$O(1)$	$O(1)$	Checks full, increments <code>top</code> , writes one value
<code>pop()</code>	$O(1)$	$O(1)$	Checks empty, reads one value, decrements <code>top</code>
<code>peek()</code>	$O(1)$	$O(1)$	Uses direct array index formula

Operation	Time Complexity	Auxiliary Space	Reason
<code>stackTop()</code>	$O(1)$	$O(1)$	Directly reads <code>S[top]</code>
<code>isEmpty()</code>	$O(1)$	$O(1)$	Checks whether <code>top == -1</code>
<code>isFull()</code>	$O(1)$	$O(1)$	Checks whether <code>top == size - 1</code>

Why Are These Operations $O(1)$?

They do not scan the whole stack.

For example, `pop()` does not search for the top value.

It already knows where the top is:

```
top
```

So it directly accesses:

```
S[top]
```

Direct array access is constant time.

That is why `pop()` is $O(1)$.

The same idea applies to `push()`, `peek()`, and `stackTop()`.

23. Overflow and Underflow

Overflow

Overflow happens when we try to push into a full stack.

Full condition:

```
top == size - 1
```

Example:

```
size = 5  
top = 4
```

Call:

```
push(&st, 60);
```

This causes:

Stack Overflow

Because there is no free slot left.

Underflow

Underflow happens when we try to pop from an empty stack.

Empty condition:

```
top == -1
```

Example:

```
top = -1
```

Call:

```
pop(&st);
```

This causes:

Stack Underflow

Because there is no value to remove.

24. Physical Array vs Logical Stack After Pop

This is a common beginner confusion.

Suppose the stack is:

```
Index: 0    1    2
Value: 10   20   30

top = 2
```

Call:

```
pop(&st);
```

The returned value is:

```
30
```

After pop:

```
top = 1
```

The array memory might still look like this:

```
Index: 0    1    2
Value: 10   20   30
```

But the logical stack is only:

```
Top -> 20
      10
```

Why?

Because `top = 1`.

So only indices `0` and `1` are active.

Index `2` is ignored.

Important idea:

The value may remain physically in memory,
but it is no longer logically part of the stack.

25. Common Beginner Mistakes

Mistake 1: Forgetting to Check Overflow Before `push()`

Wrong:

```
st->top++;  
st->S[st->top] = x;
```

Problem:

If the stack is already full, this writes outside the valid array range.

Correct:

```
if (st->top == st->size - 1) {  
    printf("Stack Overflow\n");  
} else {  
    st->top++;  
    st->S[st->top] = x;  
}
```

Mistake 2: Forgetting to Check Underflow Before `pop()`

Wrong:

```
x = st->S[st->top];  
st->top--;
```

Problem:

If `top = -1`, this tries to access:

`S[-1]`

That is invalid.

Correct:

```
if (st->top == -1) {  
    printf("Stack Underflow\n");  
} else {  
    x = st->S[st->top];  
    st->top--;  
}
```

Mistake 3: Using the Wrong Peek Formula

Wrong:

`index = top - pos;`

Correct:

`index = top - pos + 1;`

The `+ 1` is necessary.

Example:

`top = 3`
`pos = 1`

Correct formula:

`index = 3 - 1 + 1`
`index = 3`

So `peek(1)` correctly returns `S[3]`, the top element.

Without `+ 1`:

```
index = 3 - 1  
index = 2
```

That would incorrectly return the second element from the top.

Mistake 4: Thinking `peek()` Removes Data

Wrong idea:

```
peek() removes the value.
```

Correct idea:

```
peek() only reads the value.  
pop() removes the value.
```

Mistake 5: Confusing `stackTop()` and `pop()`

```
stackTop() reads the top value.
```

```
pop() removes and returns the top value.
```

Example:

```
Top -> 30  
      20  
      10
```

Call:

```
stackTop(st)
```

Result:

```
returns 30
stack remains unchanged
```

Call:

```
pop(&st)
```

Result:

```
returns 30
30 is removed
top moves down
```

Mistake 6: Passing the Stack by Value to `push()` or `pop()`

Wrong:

```
void push(struct Stack st, int x)
```

Problem:

This receives a copy of the stack.

Changing `st.top` inside the function may not correctly update the original stack variable.

Correct:

```
void push(struct Stack *st, int x)
```

and call it using:

```
push(&st, 10);
```

Same for `pop()`:

```
int pop(struct Stack *st)
```

Mistake 7: Forgetting That Positions in `peek(pos)` Start from 1

In `peek(pos)`, position counting starts from the top.

```
Top -> 40   position 1
       30   position 2
       20   position 3
```

So:

```
peek(1) returns 40
```

Not `peek(0)`.

Position `0` is invalid.

26. Key Formulas and Rules

Empty Formula

```
top == -1
```

Full Formula

```
top == size - 1
```

Push Rule

```
top++;
S[top] = x;
```

Pop Rule

```
x = S[top];  
top--;  
return x;
```

Peek Formula

```
index = top - pos + 1;
```

stackTop Rule

```
return S[top];
```

Overflow

Trying to push into a full stack.

Underflow

Trying to pop from an empty stack.

27. Mini Practice Trace 1

Given:

```
size = 4  
top = -1
```

Perform:

```
push(&st, 5);  
push(&st, 10);  
push(&st, 15);
```

Trace:

Step	Operation	top Before	top After	Active Stack
0	Start	-1	-1	Empty
1	push(5)	-1	0	5
2	push(10)	0	1	5, 10
3	push(15)	1	2	5, 10, 15

Final array:

```
Index: 0   1   2   3
Value: 5   10  15  empty

top = 2
```

Logical stack:

```
Top -> 15
      10
      5
```

28. Mini Practice Trace 2

Given:

```
Index: 0   1   2
Value: 5   10  15

top = 2
```

Call:

```
pop(&st);
```

Steps:

1. top is not -1, so the stack is not empty.
2. Save $S[2] = 15$.
3. Decrease top from 2 to 1.
4. Return 15.

Result:

```
pop() returns 15
top = 1
```

Logical stack:

```
Top -> 10
       5
```

29. Mini Practice Trace 3

Given:

```
Index: 0   1   2   3
Value: 5  10  15  20

top = 3
```

Find:

```
peek(st, 2)
```

Formula:

```
index = top - pos + 1
```

Substitute:

```
index = 3 - 2 + 1
index = 2
```

So:

```
peek(2) returns S[2]
```

Since:

```
S[2] = 15
```

Answer:

```
peek(2) returns 15
```

30. Checkpoint Questions

Try answering these without looking at the answers.

1. What does `push(x)` do?
 2. What does `pop()` do?
 3. What does `peek(pos)` do?
 4. What does `stackTop()` do?
 5. What does `isEmpty()` check?
 6. What does `isFull()` check?
 7. What is the empty condition in an array stack?
 8. What is the full condition in an array stack?
 9. What is stack overflow?
 10. What is stack underflow?
 11. Why does `push()` receive `struct Stack *st`?
 12. Why does `pop()` receive `struct Stack *st`?
 13. Why can `peek()` receive `struct Stack st`?
 14. What is the formula for `peek(pos)`?
 15. If `top = 3`, what array index does `peek(1)` access?
 16. If `top = 3`, what array index does `peek(4)` access?
 17. Does `stackTop()` remove the top value?
 18. Does `peek()` remove any value?
 19. Why are array stack operations usually `O(1)`?
 20. What is the difference between physical array content and logical stack content after `pop()`?
-

31. Checkpoint Answers

1. `push(x)` inserts `x` at the top of the stack.
 2. `pop()` removes and returns the top element.
 3. `peek(pos)` reads an element at a position from the top without deleting it.
 4. `stackTop()` reads the current top element without deleting it.
 5. `isEmpty()` checks whether `top == -1`.
 6. `isFull()` checks whether `top == size - 1`.
 7. Empty condition: `top == -1`.
 8. Full condition: `top == size - 1`.
 9. Stack overflow means trying to push into a full stack.
 10. Stack underflow means trying to pop from an empty stack.
 11. `push()` receives `struct Stack *st` because it changes the original stack.
 12. `pop()` receives `struct Stack *st` because it changes `top` in the original stack.
 13. `peek()` can receive `struct Stack st` because it only reads from the stack.
 14. Peek formula: `index = top - pos + 1`.
 15. If `top = 3`, `peek(1)` accesses index `3`.
 16. If `top = 3`, `peek(4)` accesses index `0`.
 17. No. `stackTop()` does not remove the top value.
 18. No. `peek()` only reads a value.
 19. They are `O(1)` because they directly use `top` and do not scan the whole array.
 20. After `pop()`, the removed value may still exist physically in memory, but it is ignored because `top` has moved down.
-

32. Practice Tasks

Task 1

Implement these functions:

```
push()
pop()
peek()
stackTop()
isEmpty()
isFull()
```

Use this structure:

```
struct Stack {
    int size;
```

```
int top;  
int *S;  
};
```

Task 2

Dry-run this sequence:

```
push(&st, 10);  
push(&st, 20);  
push(&st, 30);  
pop(&st);  
push(&st, 40);  
stackTop(st);
```

Start with:

```
size = 5  
top = -1
```

Write the value of `top` after every operation.

Task 3

Given:

```
Index: 0    1    2    3  
Value: 10   20   30   40  
  
top = 3
```

Find:

```
peek(1)  
peek(2)  
peek(3)  
peek(4)  
peek(5)
```

Use:

```
index = top - pos + 1
```

Task 4

Make a table showing whether each operation changes the stack.

Use this format:

Operation	Changes Stack?	Reason
push()		
pop()		
peek()		
stackTop()		
isEmpty()		
isFull()		

Task 5

Write test cases for:

- Normal push.
- Stack overflow.
- Normal pop.
- Stack underflow.
- Valid peek.
- Invalid peek.

33. Short Glossary

Term	Simple Meaning
Stack	A data structure that follows LIFO
LIFO	Last In, First Out

Term	Simple Meaning
Array stack	A stack implemented using an array
<code>size</code>	Maximum stack capacity
<code>top</code>	Index of the current top element
<code>S</code>	Pointer to stack array storage
<code>push(x)</code>	Insert <code>x</code> at the top
<code>pop()</code>	Remove and return the top element
<code>peek(pos)</code>	Read a position from the top without deleting
<code>stackTop()</code>	Read the current top element only
<code>isEmpty()</code>	Check whether the stack has no active elements
<code>isFull()</code>	Check whether the stack has no free space
Overflow	Trying to push into a full stack
Underflow	Trying to pop from an empty stack
Pass by pointer	Passing an address so the function can modify the original structure
Pass by value	Passing a copy, useful when the function only reads
<code>O(1)</code>	Constant time; work does not grow with stack size

34. Final Lesson 3 Summary

Lesson 3 teaches the main operations of an array stack.

The stack structure is:

```
struct Stack {
    int size;
    int top;
    int *S;
};
```

The most important formulas are:

```
Empty: top == -1
Full:  top == size - 1
Peek:  index = top - pos + 1
```

A successful `push()` does this:

```
top++;
S[top] = x;
```

A successful `pop()` does this:

```
x = S[top];
top--;
return x;
```

`peek(pos)` reads a value at a position from the top without removing it.

`stackTop()` reads only the current top element.

`isEmpty()` checks whether the stack is empty.

`isFull()` checks whether the stack is full.

`push()` and `pop()` change the stack, so they receive the stack by pointer:

```
struct Stack *st
```

`peek()`, `stackTop()`, `isEmpty()`, and `isFull()` only read the stack, so they can receive the stack by value:

```
struct Stack st
```

All Lesson 3 operations are `O(1)` because they work directly using `top` and do not scan the whole stack.

The most important idea to remember is:

```
top controls the array stack.
```

If you can correctly update and understand `top`, you can understand array stack operations.